다이스 POS 기말 프로젝트

공통 개발 가이드

팀 전원이 같이 지키는 개발 환경 · 구조 · 공통 클래스 · DB · 규칙 · 일정

개인 개발 문서는 "내가 만들 것", 이 문서는 "다 같이 맞출 것"입니다. **Java + MySQL 만** 사용합니다(외부 라이브러리는 MySQL 연결 드라이버 하나). 이 문서의 내용을 바꿀 때는 반드시 팀에 먼저 알리세요.

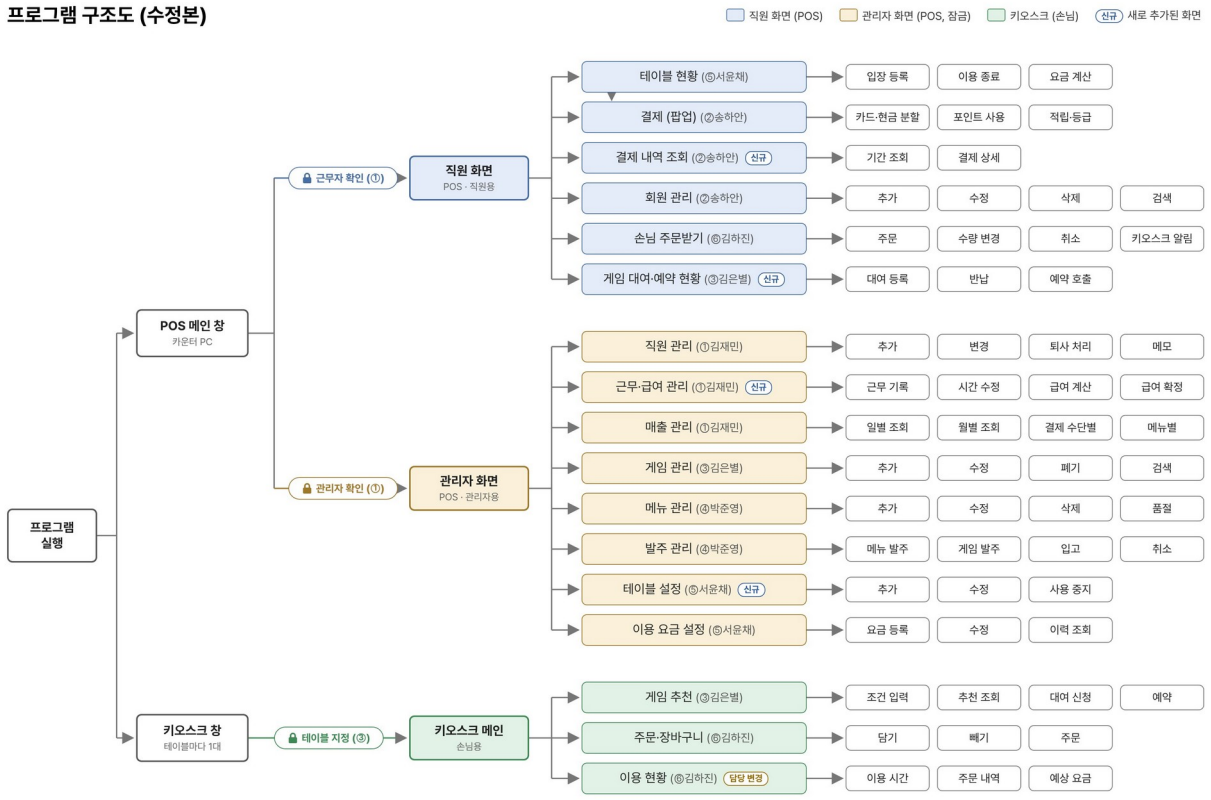
1. 개발 환경

항목	버전 / 이름	비고
IDE	Eclipse IDE for Java Developers (최신)	팀 전원 같은 IDE
JDK	JDK 17	Eclipse 프로젝트 설정에서 Java 17 선택
DBMS	MySQL Community Server 8.0	설치할 때 root 비밀번호를 기억해 둘 것
DB 도구	MySQL Workbench 8.0	공통 SQL 실행, 데이터 확인용
JDBC 드라이버	mysql-connector-j (8.x) jar	**유일한 외부 라이브러리**. 프로젝트 Build Path 에 추가
화면	Java Swing (javax.swing), 기본 Look and Feel	9 주차 예시처럼 기본 모양. 그래프도 직접 그림
암호화	java.security.MessageDigest (SHA-256)	Java 기본 기능

처음 설정 순서

1. MySQL 8.0 과 Workbench 설치 → Workbench 로 접속
2. 공통 SQL 파일(dicepos_schema.sql)을 열어 전체 실행 → dicepos DB 와 17 개 테이블, 예시 데이터 생성
3. Workbench 에서 팀 공용 계정을 만듦: 사용자 dicepos / 비밀번호 1234, dicepos DB 모든 권한 (DBUtil 이 이 계정으로 접속)
4. Eclipse 에서 Java 프로젝트 "DicePOS" 생성(JDK 17), module-info.java 는 만들지 않음
5. mysql-connector-j jar 를 프로젝트 lib 폴더에 넣고 Build Path → Add to Build Path
6. GitHub 저장소를 받아(clone) 4 장 패키지 구조 그대로 사용

2. 전체 구조



[그림 1] 프로그램 구조도 (수정본 계획서와 같음)

창은 두 개입니다. **POS 메인 창**(카운터 PC, PosMain 으로 실행)과 **키오스크 창**(테이블마다, KioskMain 으로 실행). 둘 다 같은 dicepos DB 에 연결합니다. 실습할 때는 한 PC 에서 두 프로그램을 같이 켜면 됩니다.

3. 패키지와 클래스 이름 규칙

패키지	담당	들어가는 클래스 (예)
dicepos.common	①④⑤	DBUtil, UIUtil (④) · Session, PasswordUtil, AuthDialog (①) · FeeCalculator (⑤)
dicepos.main	④③	PosMain, PosMainFrame (④) · KioskMain, KioskFrame (③)
dicepos.staff	①	StaffPanel, StaffDialog, MemoDialog, WorkSalaryPanel, SalesPanel, StaffDAO, WorkLogDAO, SalaryDAO, SalesDAO, StaffDTO …
dicepos.member / dicepos.payment	②	MemberPanel, MemberDAO, MemberDTO / PaymentDialog, PaymentHistoryPanel, PaymentDAO, PaymentDTO, PaymentDetailDTO
dicepos.game	③	GamePanel, GameRentalPanel, GameRecommendPanel, GameCard, KioskStartPanel, KioskHomePanel, GameDAO, RecommendDAO, GameRentalDAO …
dicepos.menu	④	MenuPanel, PurchaseOrderPanel, ReceiveDialog, MenuDAO, PurchaseOrderDAO, MenuDTO …
dicepos.table	⑤	TableStatusPanel, TableCard, UsageStartDialog, UsageEndDialog, TableSettingPanel, FeeSettingPanel, CafeTableDAO, TableUsageDAO, FeeSettingDAO …
dicepos.order	⑥	OrderPanel, MenuButtonListener, KioskOrderPanel, MenuCard, OrderConfirmDialog, KioskUsagePanel, OrderDAO, OrderDTO …

이름 규칙

- **화면** : XxxPanel (메인 창 가운데에 들어가는 화면), XxxDialog (팝업), 키오스크 화면도 XxxPanel
- **DB** : XxxDAO (SQL 실행), XxxDTO (테이블 한 줄을 담는 객체). DTO 를 붙이는 이유: Menu 는 java.awt.Menu 와 이름이 겹침
- **DB 이름** : 테이블·필드는 계획서 그대로 소문자_밑줄 (예: pay_time). Java 필드는 payTime 처럼 바꿔 써도 됨
- **메서드** : 조회 findXxx, 등록 insert, 수정 update, 삭제 delete. 다른 팀원이 부르는 메서드는 개인 문서의 이름 그대로

4. 공통 클래스

클래스	담당	하는 일	주요 메서드
DBUtil	④	DB 접속 정보(URL·계정)를 한 곳에 두고 연결을 만들어 줌	getConnection(), close(...)
UIUtil	④	경고창·확인창·금액 표시 형식 통일	warn(부모, 문구), confirm(부모, 문구), won(금액)
PosMainFrame	④	POS 메인 창. 왼쪽 메뉴 + CardLayout. 팀원 화면을 등록	addScreen(이름, 화면, 관리자여부), show(이름)
Session	①	지금 근무 중인 직원, 관리자 잠금 상태, 자동 잠금	getStaffId(), getName(), isAdmin(), isAdminUnlocked(), unlockAdmin(), lock()
PasswordUtil	①	비밀번호 SHA-256 변환	sha256(평문)
AuthDialog	①	근무자 확인(출근)·관리자 확인 팝업	showClockIn(부모), showAdminCheck(부모) → boolean
FeeCalculator	⑤	시간 요금 계산	perPerson(분), total(분, 인원), minutesBetween(시작, 끝)
KioskFrame	③	키오스크 창. CardLayout, 이 키오스크의 테이블 번호	show(이름), getTableNo()

공통 클래스는 **10 주차 안에 먼저** 만들어 GitHub 에 올립니다. 다른 팀원이 이걸 기다리므로, 처음에는 메서드 이름과 간단한 동작만 있어도 됩니다.

5. 팀원끼리 맞물리는 부분 (연동 지도)

다른 팀원의 테이블을 바꿔야 할 때는 **직접 SQL 을 쓰지 않고 담당자의 메서드를 부릅니다.** 트랜잭션 안에서 부르는 메서드는 **Connection 을 매개변수로 받아서**, 부른 쪽의 트랜잭션에 함께 묶입니다.

메서드	제공	사용	언제
TableUsageDAO.finish(Connection, usageId)	⑤	②	결제 트랜잭션 안에서 이용 상태 '결제완료'
FeeCalculator.perPerson / total	⑤	②⑥	결제 이용료, 키오스크 이용 현황
TableUsageDAO.findActiveByTable / findActiveAll	⑤	③⑥	이 테이블이 이용 중인지, 주문받기 테이블 콤보
CafeTableDAO.findActive()	⑤	③	키오스크 테이블 지정
OrderDAO.findDetailsByUsage(usageId)	⑥	②	결제할 메뉴 목록
OrderDAO.hasMenuOrder(menuId)	⑥	④	메뉴 삭제 전 확인
MenuDAO.decreaseStock / increaseStock(Connection, ...)	④	⑥	주문·취소 트랜잭션 안에서 재고
MenuDAO.findByCategory(분류)	④	⑥	메뉴 버튼·카드

메서드	제공	사용	언제
GameDAO.increaseStock(Connection, ...), findAll()	③	④	보드게임 발주·입고
GameRentalDAO.countRenting(usageId)	③	⑤	이용 종료 시 미반납 게임 확인
MemberDAO.search(검색어)	②	⑤	이용 시작 회원 검색
new PaymentDialog(부모, usageId)	②	⑤	이용 종료 → 결제
Session / AuthDialog / PasswordUtil	①	전원	근무자·관리자 확인, 처리 직원 번호
DBUtil, UIUtil, PosMainFrame.addScreen	④	전원	DB 연결, 경고창, 메인 창 등록
KioskFrame.show / getTableNo	③	⑥	키오스크 화면 전환

6. 공통 규칙

6.1 DB 연결과 트랜잭션

1. DB 연결은 항상 DBUtil.getConnection()으로만 얻는다 (접속 정보가 한 곳에만 있게).
2. 쓰고 나면 반드시 닫는다 (try-with-resources 또는 finally).
3. SQL 에 값을 넣을 때는 문자열을 이어 붙이지 않고 PreparedStatement 의 ? 자리에 넣는다 (SQL 인젝션·따옴표 오류 방지).
4. 여러 테이블을 한 번에 바꾸는 작업은 트랜잭션: 자동 커밋 끄기 → 작업들 → 모두 성공하면 commit → 예외가 나면 rollback → 마지막에 자동 커밋 다시 켜기.
5. 트랜잭션 안에서 다른 팀원 메서드를 부를 때는 내 Connection 을 넘긴다. 넘겨받은 쪽은 commit·rollback·close 를 하지 않는다.
6. 헤더 + 상세를 저장할 때는 헤더 INSERT 후 자동 생성된 번호(getGeneratedKeys)를 받아 상세에 넣는다.

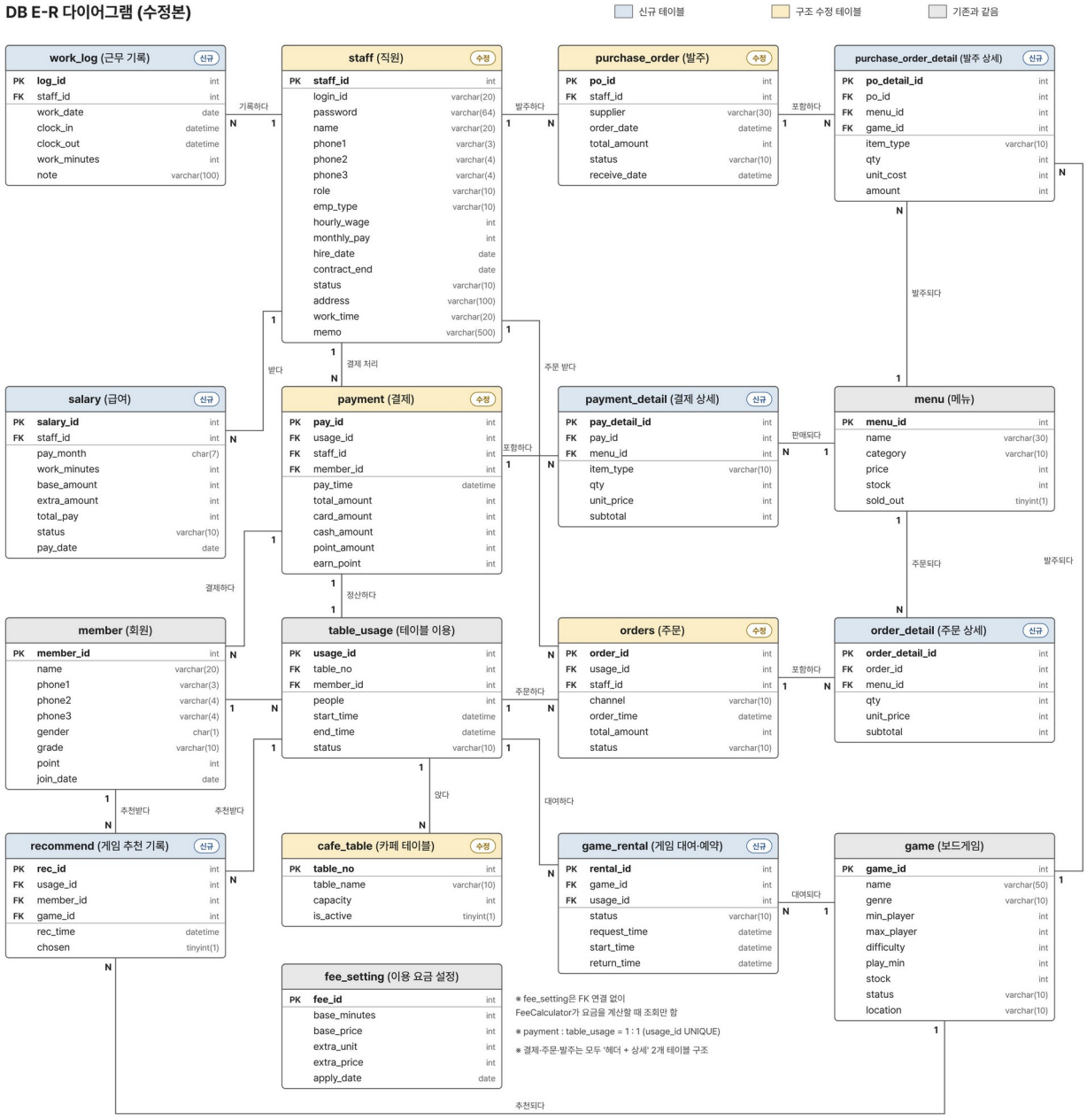
6.2 화면

- 화면 크기: POS 메인 창 1280 × 800, 키오스크 창 1024 × 768 (팝업은 자유).
- 글꼴: 맑은 고딕으로 통일 (UIUtil 에서 한 번에 설정).
- 9 주차 예시처럼 화면을 P 영역으로 나누고, **선택하지 않고 누르면 안 되는 버튼은 꺼 둔다** (setEnabled(false)). 그래도 눌린 경우를 대비해 경고창도 둔다.
- 경고·확인은 UIUtil.warn / confirm 으로 통일 (제목·아이콘이 같아 보이게).
- 금액은 int 로 저장하고 화면에는 UIUtil.won()으로 "24,500 원"처럼 표시.
- 날짜·시각은 DB 에서 DATE / DATETIME, Java 에서는 LocalDate / LocalDateTime 사용.
- 스레드·타이머에서 화면을 바꿀 때는 SwingUtilities.invokeLater 안에서.

6.3 협업

- 각자 **자기 패키지만** 수정한다. 공통 클래스는 담당자만 고치고, 바꿀 때는 팀에 알린다.
- DB 구조(테이블·필드)를 바꾸려면 담당자가 팀에 먼저 말하고, dicepos_schema.sql 과 계획서를 같이 고친다.
- GitHub: 매일 작업 전 받기(pull), 끝나면 올리기(push). 올리기 전에 프로그램이 실행되는지 확인.
- 매주 회의에서 6 장 일정 대비 진행 상황을 공유한다.

7. DB



[그림 2] DB E-R 다이어그램 (17 개 테이블)

테이블	이름	담당	종류
staff	직원	① 김재민	
work_log	근무 기록	① 김재민	
salary	급여	① 김재민	
member	회원	② 송하안	
payment	결제	② 송하안	헤더 (1 건 전체)
payment_detail	결제 상세	② 송하안	상세 (품목마다 1 줄)
game	보드게임	③ 김은별	
recommend	게임 추천 기록	③ 김은별	
game_rental	게임 대여·예약	③ 김은별	

테이블	이름	담당	종류
menu	메뉴	④ 박준영	
purchase_order	발주	④ 박준영	헤더 (1 건 전체)
purchase_order_detail	발주 상세	④ 박준영	상세 (품목마다 1 줄)
cafe_table	카페 테이블	⑤ 서윤채	
table_usage	테이블 이용	⑤ 서윤채	
fee_setting	이용 요금 설정	⑤ 서윤채	
orders	주문	⑥ 김하진	헤더 (1 건 전체)
order_detail	주문 상세	⑥ 김하진	상세 (품목마다 1 줄)

DB 생성 스크립트 (dicepos_schema.sql)

아래 SQL 은 **DB 설계를 그대로 옮긴 것**으로, 팀 전원이 같은 DB 를 만들기 위해 사용합니다. GitHub 저장소의 dicepos_schema.sql 파일과 같으며 Workbench 에서 열어 전체 실행하면 됩니다. MySQL 호환 DB 에서 실행해 17 개 테이블과 예시 데이터가 오류 없이 만들어지는 것을 확인했습니다. 예시 데이터 계정: **admin / admin1234 (관리자)**, **minseo / staff1234 (직원)**.

```
-- ============================================================
-- 다이스 POS (dicepos) DB 생성 스크립트 – MySQL 8.0
-- 피드백 반영 수정본 계획서의 17개 테이블과 같은 구조
-- 실행: MySQL Workbench에서 이 파일을 열고 전체 실행(번개 아이콘)
-- ============================================================
DROP DATABASE IF EXISTS dicepos;
CREATE DATABASE dicepos DEFAULT CHARACTER SET utf8mb4 COLLATE utf8mb4_general_ci;
USE dicepos;

-- staff (직원) : 담당 ①
CREATE TABLE staff (
  staff_id         INT           NOT NULL AUTO_INCREMENT COMMENT '직원 번호',
  login_id         VARCHAR(20)   NOT NULL COMMENT '아이디 (중복 불가)',
  password         VARCHAR(64)   NOT NULL COMMENT '비밀번호 (SHA-256 으로 암호화한 값만 저장)',
  name             VARCHAR(20)   NOT NULL COMMENT '이름',
  phone1           VARCHAR(3)    NOT NULL COMMENT '전화번호 1',
  phone2           VARCHAR(4)    NOT NULL COMMENT '전화번호 2',
  phone3           VARCHAR(4)    NOT NULL COMMENT '전화번호 3',
  role             VARCHAR(10)   NOT NULL DEFAULT '직원' COMMENT '권한 (관리자 / 직원). 관리자만 관리자 화면을 열 수
있음',
  emp_type         VARCHAR(10)   NOT NULL COMMENT '고용 형태 (정규직 / 계약직 / 시간제)',
  hourly_wage      INT           COMMENT '시급 (시간제만 입력, 나머지는 NULL)',
  monthly_pay      INT           COMMENT '월 기본급 (정규직·계약직만 입력)',
  hire_date        DATE          NOT NULL COMMENT '입사일',
  contract_end     DATE          COMMENT '계약 종료일 (계약직만 입력)',
  status           VARCHAR(10)   NOT NULL DEFAULT '재직' COMMENT '재직 상태 (재직 / 퇴사)',
  address          VARCHAR(100)  COMMENT '주소 (선택 입력)',
  work_time        VARCHAR(20)   COMMENT '기본 근무 시간대 (예: 09:00~18:00, 선택 입력)',
  memo             VARCHAR(500)  COMMENT '직원 메모 (선택 입력)',
  PRIMARY KEY (staff_id),
  UNIQUE (login_id)
) COMMENT '직원';

-- member (회원) : 담당 ②
CREATE TABLE member (
  member_id        INT           NOT NULL AUTO_INCREMENT COMMENT '회원 번호',
  name             VARCHAR(20)   NOT NULL COMMENT '이름',
  phone1           VARCHAR(3)    NOT NULL COMMENT '전화번호 1',
```

```
  phone2           VARCHAR(4)    NOT NULL COMMENT '전화번호 2',
  phone3           VARCHAR(4)    NOT NULL COMMENT '전화번호 3',
  gender           CHAR(1)       NOT NULL COMMENT '성별 (M / F)',
  grade            VARCHAR(10)   NOT NULL DEFAULT '일반' COMMENT '등급 (일반 / 실버 / 골드), 결제할 때 자동 변경',
  point            INT           NOT NULL DEFAULT 0 COMMENT '보유 포인트, 기본값 0',
  join_date        DATE          NOT NULL COMMENT '가입일',
  PRIMARY KEY (member_id),
  UNIQUE (phone1, phone2, phone3)
) COMMENT '회원';

-- cafe_table (카페 테이블) : 담당 ⑤
CREATE TABLE cafe_table (
  table_no         INT           NOT NULL COMMENT '테이블 번호',
  table_name       VARCHAR(10)   NOT NULL COMMENT '표시 이름 (예: T1, 룸A)',
  capacity         INT           NOT NULL COMMENT '최대 좌석 수 (인원 초과 검사)',
  is_active        TINYINT(1)    NOT NULL DEFAULT 1 COMMENT '사용 여부 (0이면 테이블 현황에 표시하지 않음)',
  PRIMARY KEY (table_no),
  UNIQUE (table_name)
) COMMENT '카페 테이블';

-- fee_setting (이용 요금 설정) : 담당 ⑤
CREATE TABLE fee_setting (
  fee_id           INT           NOT NULL AUTO_INCREMENT COMMENT '요금 번호',
  base_minutes     INT           NOT NULL COMMENT '기본 시간(분)',
  base_price       INT           NOT NULL COMMENT '1인 기본요금',
  extra_unit       INT           NOT NULL COMMENT '추가 요금 단위(분)',
  extra_price      INT           NOT NULL COMMENT '1인 추가요금',
  apply_date       DATE          NOT NULL COMMENT '적용 시작일',
  PRIMARY KEY (fee_id)
) COMMENT '이용 요금 설정';

-- menu (메뉴) : 담당 ④
CREATE TABLE menu (
  menu_id          INT           NOT NULL AUTO_INCREMENT COMMENT '메뉴 번호',
  name             VARCHAR(30)   NOT NULL COMMENT '메뉴 이름 (중복 불가)',
  category         VARCHAR(10)   NOT NULL COMMENT '분류 (음료 / 간식 / 식사)',
  price            INT           NOT NULL COMMENT '판매 가격',
  stock            INT           NOT NULL DEFAULT 0 COMMENT '재고 수량, 기본값 0',
  sold_out         TINYINT(1)    NOT NULL DEFAULT 0 COMMENT '품절 여부 (1이면 주문 불가)',
  PRIMARY KEY (menu_id),
  UNIQUE (name)
) COMMENT '메뉴';

-- game (보드게임) : 담당 ③
CREATE TABLE game (
  game_id          INT           NOT NULL AUTO_INCREMENT COMMENT '게임 번호',
  name             VARCHAR(50)   NOT NULL COMMENT '게임 이름',
  genre            VARCHAR(10)   NOT NULL COMMENT '장르 (파티 / 가족 / 전략 / 추리 / 카드)',
  min_player       INT           NOT NULL COMMENT '최소 인원',
  max_player       INT           NOT NULL COMMENT '최대 인원',
  difficulty       INT           NOT NULL COMMENT '난이도 1~5 (★로 표시)',
  play_min         INT           NOT NULL COMMENT '평균 플레이 시간(분)',
  stock            INT           NOT NULL DEFAULT 0 COMMENT '보유 수량, 기본값 0 (입고하면 증가, 폐기하면 감소)',
  status           VARCHAR(10)   NOT NULL DEFAULT '정상' COMMENT '상태 (정상 / 부품누락 / 폐기)',
  location         VARCHAR(10)   COMMENT '진열 위치 (예: A-1)',
  PRIMARY KEY (game_id)
) COMMENT '보드게임';

-- table_usage (테이블 이용) : 담당 ⑤
CREATE TABLE table_usage (
```

```
  usage_id         INT           NOT NULL AUTO_INCREMENT COMMENT '이용 번호',
  table_no         INT           NOT NULL COMMENT '이용한 테이블',
  member_id        INT           COMMENT '회원 번호 (비회원은 NULL)',
  people           INT           NOT NULL COMMENT '이용 인원',
  start_time       DATETIME      NOT NULL COMMENT '입장 시각',
  end_time         DATETIME      COMMENT '퇴장 시각 (이용 중이면 NULL)',
  status           VARCHAR(10)   NOT NULL DEFAULT '이용중' COMMENT '상태 (이용중 / 결제완료)',
  PRIMARY KEY (usage_id),
  FOREIGN KEY (table_no) REFERENCES cafe_table(table_no),
  FOREIGN KEY (member_id) REFERENCES member(member_id) ON DELETE SET NULL
) COMMENT '테이블 이용';

-- work_log (근무 기록) : 담당 ①
CREATE TABLE work_log (
  log_id           INT           NOT NULL AUTO_INCREMENT COMMENT '근무 기록 번호',
  staff_id         INT           NOT NULL COMMENT '근무한 직원',
  work_date        DATE          NOT NULL COMMENT '근무 날짜',
  clock_in         DATETIME      NOT NULL COMMENT '출근 시각',
  clock_out        DATETIME      COMMENT '퇴근 시각 (근무 중이면 NULL)',
  work_minutes     INT           COMMENT '그날 근무시간(분). 퇴근할 때 계산해서 저장',
  note             VARCHAR(100)  COMMENT '비고 (관리자가 시간을 고친 경우 그 사유)',
  PRIMARY KEY (log_id),
  FOREIGN KEY (staff_id) REFERENCES staff(staff_id)
) COMMENT '근무 기록';

-- salary (급여) : 담당 ①
CREATE TABLE salary (
  salary_id        INT           NOT NULL AUTO_INCREMENT COMMENT '급여 번호',
  staff_id         INT           NOT NULL COMMENT '급여를 받는 직원',
  pay_month        CHAR(7)       NOT NULL COMMENT '급여 월 (예: 2026-11). 같은 달 중복 확정 불가',
  work_minutes     INT           NOT NULL COMMENT '그 달 누적 근무시간(분) = work_log 합계',
  base_amount      INT           NOT NULL COMMENT '기본 급여 (시간제: 누적 시간 × 시급, 정규·계약: 월 기본급)',
  extra_amount     INT           NOT NULL DEFAULT 0 COMMENT '추가 수당 (주휴수당 등, 관리자 입력, 기본값 0)',
  total_pay        INT           NOT NULL COMMENT '지급액 (base_amount + extra_amount)',
  status           VARCHAR(10)   NOT NULL DEFAULT '미지급' COMMENT '지급 상태 (미지급 / 지급완료)',
  pay_date         DATE          COMMENT '지급일 (지급 전이면 NULL)',
  PRIMARY KEY (salary_id),
  FOREIGN KEY (staff_id) REFERENCES staff(staff_id),
  UNIQUE (staff_id, pay_month)
) COMMENT '급여';

-- payment (결제) : 담당 ②
CREATE TABLE payment (
  pay_id           INT           NOT NULL AUTO_INCREMENT COMMENT '결제 번호',
  usage_id         INT           NOT NULL COMMENT '결제한 테이블 이용 건 (한 이용 건은 한 번만 결제)',
  staff_id         INT           COMMENT '결제를 처리한 직원',
  member_id        INT           COMMENT '결제한 회원 (비회원은 NULL)',
  pay_time         DATETIME      NOT NULL COMMENT '결제 일시 (연월일 시분초)',
  total_amount     INT           NOT NULL COMMENT '총 결제 금액 (= 결제 상세 소계의 합)',
  card_amount      INT           NOT NULL DEFAULT 0 COMMENT '카드 결제 금액',
  cash_amount      INT           NOT NULL DEFAULT 0 COMMENT '현금 결제 금액',
  point_amount     INT           NOT NULL DEFAULT 0 COMMENT '포인트 결제 금액 (사용한 포인트)',
  earn_point       INT           NOT NULL DEFAULT 0 COMMENT '이번 결제로 적립한 포인트 (비회원은 0)',
  PRIMARY KEY (pay_id),
  FOREIGN KEY (usage_id) REFERENCES table_usage(usage_id),
  UNIQUE (usage_id),
  FOREIGN KEY (staff_id) REFERENCES staff(staff_id) ON DELETE SET NULL,
  FOREIGN KEY (member_id) REFERENCES member(member_id) ON DELETE SET NULL
) COMMENT '결제';
```

```
-- payment_detail (결제 상세) : 담당 ②
CREATE TABLE payment_detail (
  pay_detail_id    INT           NOT NULL AUTO_INCREMENT COMMENT '결제 상세 번호',
  pay_id           INT           NOT NULL COMMENT '어느 결제의 상세인지',
  item_type        VARCHAR(10)   NOT NULL COMMENT '항목 구분 (이용료 / 메뉴)',
  menu_id          INT           COMMENT '상품(메뉴) 번호. 이용료 줄은 NULL',
  qty              INT           NOT NULL COMMENT '구매 수량 (이용료 줄은 이용 인원)',
  unit_price       INT           NOT NULL COMMENT '구매 단가 (메뉴는 menu.price에서 불러옴, 이용료는 1인 요금)',
  subtotal         INT           NOT NULL COMMENT '상품별 결제 금액 소계 (qty × unit_price)',
  PRIMARY KEY (pay_detail_id),
  FOREIGN KEY (pay_id) REFERENCES payment(pay_id) ON DELETE CASCADE,
  FOREIGN KEY (menu_id) REFERENCES menu(menu_id)
) COMMENT '결제 상세';

-- orders (주문) : 담당 ⑥
CREATE TABLE orders (
  order_id         INT           NOT NULL AUTO_INCREMENT COMMENT '주문 번호',
  usage_id         INT           NOT NULL COMMENT '어느 테이블 이용 건의 주문인지 (후불제)',
  staff_id         INT           COMMENT '주문을 받은 직원 (키오스크 주문은 NULL)',
  channel          VARCHAR(10)   NOT NULL COMMENT '주문 경로 (POS / 키오스크)',
  order_time       DATETIME      NOT NULL COMMENT '주문 일시',
  total_amount     INT           NOT NULL DEFAULT 0 COMMENT '주문 금액 합계 (= 주문 상세 소계의 합)',
  status           VARCHAR(10)   NOT NULL DEFAULT '확인' COMMENT '상태 (접수 / 확인). 키오스크 주문은 ''접수''로
들어오고 직원이 확인하면 ''확인''',
  PRIMARY KEY (order_id),
  FOREIGN KEY (usage_id) REFERENCES table_usage(usage_id),
  FOREIGN KEY (staff_id) REFERENCES staff(staff_id) ON DELETE SET NULL
) COMMENT '주문';

-- order_detail (주문 상세) : 담당 ⑥
CREATE TABLE order_detail (
  order_detail_id  INT           NOT NULL AUTO_INCREMENT COMMENT '주문 상세 번호',
  order_id         INT           NOT NULL COMMENT '어느 주문의 상세인지',
  menu_id          INT           NOT NULL COMMENT '주문한 메뉴',
  qty              INT           NOT NULL COMMENT '주문 수량',
  unit_price       INT           NOT NULL COMMENT '주문 당시 단가 (menu.price에서 불러옴)',
  subtotal         INT           NOT NULL COMMENT '소계 (qty × unit_price)',
  PRIMARY KEY (order_detail_id),
  FOREIGN KEY (order_id) REFERENCES orders(order_id) ON DELETE CASCADE,
  FOREIGN KEY (menu_id) REFERENCES menu(menu_id)
) COMMENT '주문 상세';

-- purchase_order (발주) : 담당 ④
CREATE TABLE purchase_order (
  po_id            INT           NOT NULL AUTO_INCREMENT COMMENT '발주 번호',
  staff_id         INT           COMMENT '발주한 직원',
  supplier         VARCHAR(30)   COMMENT '거래처 (선택 입력)',
  order_date       DATETIME      NOT NULL COMMENT '발주 일시',
  total_amount     INT           NOT NULL DEFAULT 0 COMMENT '총 발주 금액 (= 발주 상세 금액의 합)',
  status           VARCHAR(10)   NOT NULL DEFAULT '요청' COMMENT '상태 (요청 / 입고 / 취소)',
  receive_date     DATETIME      COMMENT '입고 일시 (입고 전이면 NULL)',
  PRIMARY KEY (po_id),
  FOREIGN KEY (staff_id) REFERENCES staff(staff_id) ON DELETE SET NULL
) COMMENT '발주';

-- purchase_order_detail (발주 상세) : 담당 ④
CREATE TABLE purchase_order_detail (
  po_detail_id     INT           NOT NULL AUTO_INCREMENT COMMENT '발주 상세 번호',
  po_id            INT           NOT NULL COMMENT '어느 발주의 상세인지',
  item_type        VARCHAR(10)   NOT NULL COMMENT '품목 구분 (메뉴 / 보드게임)',
```

```
  menu_id          INT           COMMENT '발주한 메뉴 (보드게임이면 NULL)',
  game_id          INT           COMMENT '발주한 보드게임 (메뉴면 NULL)',
  qty              INT           NOT NULL COMMENT '발주 수량',
  unit_cost        INT           NOT NULL COMMENT '매입 단가 (같은 품목의 직전 발주 단가를 불러오고 수정 가능)',
  amount           INT           NOT NULL COMMENT '발주 금액 (qty × unit_cost)',
  PRIMARY KEY (po_detail_id),
  FOREIGN KEY (po_id) REFERENCES purchase_order(po_id) ON DELETE CASCADE,
  FOREIGN KEY (menu_id) REFERENCES menu(menu_id),
  FOREIGN KEY (game_id) REFERENCES game(game_id)
) COMMENT '발주 상세';

-- recommend (게임 추천 기록) : 담당 ③
CREATE TABLE recommend (
  rec_id           INT           NOT NULL AUTO_INCREMENT COMMENT '추천 번호',
  usage_id         INT           NOT NULL COMMENT '추천을 받은 테이블 이용 건',
  member_id        INT           COMMENT '추천받은 회원 (비회원은 NULL)',
  game_id          INT           NOT NULL COMMENT '추천한 게임',
  rec_time         DATETIME      NOT NULL COMMENT '추천한 시각',
  chosen           TINYINT(1)    NOT NULL DEFAULT 0 COMMENT '손님이 이 게임을 골랐는지 (대여 신청·예약하면 1)',
  PRIMARY KEY (rec_id),
  FOREIGN KEY (usage_id) REFERENCES table_usage(usage_id),
  FOREIGN KEY (member_id) REFERENCES member(member_id) ON DELETE SET NULL,
  FOREIGN KEY (game_id) REFERENCES game(game_id)
) COMMENT '게임 추천 기록';

-- game_rental (게임 대여·예약) : 담당 ③
CREATE TABLE game_rental (
  rental_id        INT           NOT NULL AUTO_INCREMENT COMMENT '대여 번호',
  game_id          INT           NOT NULL COMMENT '대여·예약한 게임',
  usage_id         INT           NOT NULL COMMENT '어느 테이블 이용 건인지',
  status           VARCHAR(10)   NOT NULL COMMENT '상태 (예약 / 대여중 / 반납 / 취소)',
  request_time     DATETIME      NOT NULL COMMENT '신청 시각 (예약 대기 순서의 기준)',
  start_time       DATETIME      COMMENT '대여 시작 시각 (게임을 받은 시각)',
  return_time      DATETIME      COMMENT '반납 시각 (대여 중이면 NULL)',
  PRIMARY KEY (rental_id),
  FOREIGN KEY (game_id) REFERENCES game(game_id),
  FOREIGN KEY (usage_id) REFERENCES table_usage(usage_id)
) COMMENT '게임 대여·예약';

-- ============================================================
-- 예시 데이터 (개발·시연용, 필요하면 각자 추가)
-- 비밀번호는 SHA-256 값: admin1234 / staff1234
-- ============================================================
INSERT INTO staff (login_id, password, name, phone1, phone2, phone3, role, emp_type, hourly_wage,
monthly_pay, hire_date, status) VALUES
 ('admin', 'ac9689e2272427085e35b9d3e3e8bed88cb3434828b43b86fc0596cad4c6e270', '김지우', '010', '1234',
'5678', '관리자', '정규직', NULL, 2600000, '2024-03-02', '재직'),
 ('minseo', '02defbfb8190f9d0719ef7a23da2049bd2e61442bc14021a6d8a4ae35ca334b7', '박민서', '010', '9876',
'5432', '직원', '시간제', 10320, NULL, '2026-08-15', '재직');
INSERT INTO cafe_table (table_no, table_name, capacity) VALUES
 (1,'T1',4),(2,'T2',4),(3,'T3',4),(4,'T4',2),(5,'T5',6),(6,'T6',6),(7,'T7',4),(8,'T8',4);
INSERT INTO fee_setting (base_minutes, base_price, extra_unit, extra_price, apply_date) VALUES (60, 3500,
10, 500, '2026-11-01');
INSERT INTO menu (name, category, price, stock) VALUES
 ('아메리카노','음료',3000,50),('오렌지주스','음료',3000,30),('콜라','음료',2000,24),
 ('감자칩','간식',2500,20),('츄러스','간식',3500,0),('라면','식사',4500,15),('떡볶이','식사',6000,10);
UPDATE menu SET sold_out = 1 WHERE stock = 0;
INSERT INTO game (name, genre, min_player, max_player, difficulty, play_min, stock, location) VALUES
 ('스플렌더','전략',2,4,2,30,2,'A-2'),('딕싯','가족',3,6,2,30,2,'C-2'),('우노','카드',2,10,1,20,4,'B-2'),
 ('젠가','파티',2,8,1,15,2,'C-1'),('할리갈리','카드',2,6,1,20,2,'B-1'),('카탄','전략',3,4,3,60,1,'A-1');
```

```
INSERT INTO member (name, phone1, phone2, phone3, gender, join_date) VALUES
 ('김민수','010','1234','5678','M','2026-03-01'),('이서연','010','2345','1122','F','2026-05-10');
```

8. 팀 일정 (9 주차 오리엔테이션 일정 기준)

주차	팀 목표	확인 방법
10 주차	공통 클래스(④①⑤③) 먼저 완성 → 각자 화면 배치와 이벤트(가짜 데이터)	PosMain 을 실행하면 모든 팀원 화면이 메뉴로 열림
11 주차	전원 같은 DB 구축, 각자 단일 테이블 CRUD 연동	각자 자기 테이블 C·R·U·D 시연 가능
12~13 주차	트랜잭션과 연동 메서드 완성, 스레드·타이머	아래 통합 시나리오 1~6 단계 통과
14 주차	통합 테스트, 버그 수정, 결과보고서 제출	통합 시나리오 전체 통과, 보고서 제출
15 주차	결과물 테스트 (개인 시연 + 질의응답)	각자 개인 문서 9 장 순서로 시연

통합 테스트 시나리오 — 손님 한 팀의 전체 흐름

1. [①] 직원 minseo 출근(근무자 확인) → POS 메인 창
2. [⑤] T3 에 회원 김민수 2 명 입장 등록
3. [③] T3 키오스크에서 게임 추천 → 대여 신청 / 예약
4. [⑥] T3 키오스크에서 오렌지주스 2, 라면 1 주문 → POS 에 알림 → 확인
5. [⑤→②] T3 이용 종료(미반납 게임 경고 → 반납) → 결제 팝업에서 포인트 + 카드 + 현금 나눠 결제
6. [②] 결제 내역 조회에서 결제 상세 3 줄(이용료·주스·라면) 확인, 회원 포인트 변경 확인
7. [①] 관리자 확인 → 매출 관리에 방금 결제가 집계됐는지 확인
8. [④] 라면 재고가 1 줄었는지 확인 → 발주·입고 후 재고 증가 확인
9. [①] 퇴근 → 근무·급여 관리에 오늘 근무시간 표시